

QueueUp - Smart Canteen & Pre-Order Platform

เอกสารสถาปัตยกรรมระบบ โครงสร้างฟังก์ชัน และมาตรการความปลอดภัยฉบับสมบูรณ์

1. ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

เว็บแอปพลิเคชัน QueueUp พัฒนาขึ้นด้วยสถาปัตยกรรมโมเดิร์นแบบ Single Page Application (SPA) บน React 19, Redux Toolkit, Context API ร่วมกับ Cloud Firestore และ Firebase Authentication เพื่อแก้ไขปัญหาการรอคิวในโรงอาหารสถานศึกษาอย่างมีประสิทธิภาพ โดยแบ่งเลเยอร์การทำงานออกเป็น 3 ระดับหลัก:

- Presentation & State Layer:** ควบคุมหน้า UI ทั้งหมด (Home, Product, Purchase, Profile, Merchant, Admin) ซิงค์ข้อมูลแบบ Real-time ด้วย Redux Store และ React Context
- Security & Authorization Layer:** ProtectedRoute (RBAC), Information Hiding, Salted SHA-256 Hashing, Input Sanitization, High-Entropy Account ID
- Backend & Cloud Storage Layer:** Firebase Authentication, Cloud Firestore Database และ Firestore Security Rules ที่บังคับใช้ Store Isolation

2. โครงสร้างไฟล์และหน้าที่ของแต่ละโมดูล (Directory & Module Breakdown)

โฟลเดอร์ / โมดูล	ไฟล์สำคัญ	หน้าที่และการทำงานหลัก
src/components/	ShopeeSearchBar.jsx FoodCard.jsx ProtectedRoute.jsx PaymentModal.jsx ChatModal.jsx	คอมโพเนนต์ส่วนกลางสำหรับ Header ค้นหา, การ์ดแสดงอาหารพร้อมเวลาปรุง, ระบบ Guard ตรวจสอบสิทธิ์ RBAC, ระบบชำระเงิน PromptPay และแชท Real-time
src/context/	AuthContext.jsx PreferencesContext.jsx ThemeContext.jsx	จัดการ Lifecycle การล็อกอิน Firebase + Google Sign-In, การสลับภาษาไทย/อังกฤษ (TH/EN) ทั่วทั้งระบบ และการจัดการธีมสี
src/store/	index.js authSlice.js cartSlice.js notificationSlice.js	Redux State จัดการ Session ผู้ใช้, สิทธิ์ Role, รายการในตะกร้าสินค้า, การคำนวณส่วนลดคูปอง และการแจ้งเตือน Real-time Toasts
src/pages/	Home.jsx ProductDetail.jsx UserPurchase.jsx UserProfile.jsx StoreAdminPage.tsx MerchantDashboard.jsx	หน้าเว็บหลัก: เมนูอาหารและแคตตาล็อก, หน้ารายละเอียดและเลือกรอบเวลารับ, ติดตามคิวออเดอร์, จัดการโปรไฟล์, หน้าจอ KDS ครั้ว และแดชบอร์ดแอดมินกลาง
src/services/	aiBehaviorEngine.js aiSecurityShield.js aiUserVerificationEngine.js	ระบบ AI วิเคราะห์พฤติกรรมเพื่อนำเสนอ 1-Click Quick Re-Order, ตรวจสอบสุขภาพความปลอดภัยบัญชี และคำนวณ Trust Score ของผู้ใช้งาน
Root Configs	firebase.rules vercel.json src/firebase/config.js	กฎความปลอดภัย Firestore Rules 3 ระดับ, HTTP Security Headers ป้องกัน Clickjacking/XSS และการโหลดคีย์อย่างปลอดภัย

3. ลำดับขั้นตอนการทำงานของฟังก์ชันหลัก (Key Functional Flows)

1. การสั่งอาหารและจองรอบเวลารับ (Customer Pre-Order Flow):

ลูกค้าเลือกเมนูอาหารจากหน้า /home หรือ /product/:id ☐ ระบุตัวเลือกอาหารและรอบเวลาที่ต้องการรับ (Time Slot) ☐
ระบบคำนวณเวลาปรุงและคิวแบบสด ☐ กดยืนยันชำระเงินผ่าน PromptPay QR พร้อมแนบสลิป ☐ ออเดอร์ถูกส่งเข้าคอลเลกชัน /orders ในสถานะ TO_SHIP

2. ระบบจัดการครั้วและคิวอาหาร (Merchant KDS Flow):

ร้านค้าล็อกอินเข้าสู่ /merchant/dashboard ☐ ระบบตรวจสอบความเป็นเจ้าของร้าน (Store Isolation) ☐
แสดงคิวออเดอร์ของร้านตนเองแบบ Real-time ☐ เมื่อทำอาหารเสร็จ ร้านค้ากดเปลี่ยนสถานะเป็น READY ☐
ระบบแจ้งเตือนให้ลูกค้ามารับอาหารร้อนๆ หันที่

3. ระบบบริหารจัดการส่วนกลาง (Super Admin Flow):

ผู้ดูแลระบบสูงสุด (58140@lomsak.ac.th) เข้าสู่ระบบ /admin จัดการเมนูและสต็อก, เพิ่ม/ลบรายชื่อพนักงานและสิทธิ์, สร้างแคมเปญคุ้มครองส่วนลด CRM และส่งออกรายงานยอดขายเป็นไฟล์ CSV สำหรับตรวจสอบบัญชี

4. มาตรการความปลอดภัย 4 ชั้นระดับ Enterprise (4-Tier Security Matrix)

ระดับชั้นความปลอดภัย	ฟังก์ชัน / โมดูล	กลไกและมาตรการป้องกัน
ชั้นที่ 1: Browser Headers	vercel.json	X-Frame-Options: SAMEORIGIN (ป้องกัน Clickjacking), X-Content-Type-Options: nosniff, X-XSS-Protection, HSTS บังคับ HTTPS 100%
ชั้นที่ 2: Client Guard & RBAC	ProtectedRoute.jsx	ตรวจสอบ Role (Customer, Merchant, Admin), ขอนหน้าแอตมินด้วย 404 Not Found สำหรับคนนอก, จัดจำ Pathname + Query เพื่อพากลับหน้าเดิมหลังล็อกอิน
ชั้นที่ 3: Data Sanitization	utils/security.js	ป้องกัน XSS Injection ด้วย sanitizeInput(), แฮชรหัสผ่านด้วย Salted SHA-256, สร้างรหัสบัญชีความปลอดภัยสูง High-Entropy Account ID
ชั้นที่ 4: Backend Rules	firestore.rules	Store Isolation ป้องกันร้านค้าก๊วดยข้อมูลกัน, ล็อคค่าได้เฉพาะออเดอร์ตนเอง, Audit Log ความปลอดภัยเป็นแบบ Immutable Append-Only (ห้ามลบ)